

W16 Assignment

Agentify the Assistant

AI Fellowship - Task 3

Royas Shakya

Date: 2026-09-11

1. Why a Fixed Pipeline Is Not Sufficient

A fixed pipeline cannot handle cross-source verification because the number and type of sources needed varies per query. A factual question like "What is RAG?" may only need one knowledge base lookup, while "Compare RAG vs fine-tuning for enterprise use" requires searching both the knowledge base and the web, evaluating evidence sufficiency at each step, and deciding whether to continue searching or provide an answer. This decision depends on what the model discovers mid-loop and cannot be predetermined.

2. Agentic Feature: Cross-Source Verification

The new feature is a cross-source verification loop that iteratively gathers evidence from multiple sources (knowledge base, web search) and verifies findings before providing a final answer. The LLM decides at each iteration whether to search again or provide an answer based on evidence sufficiency.

3. Agentic Loop Architecture

The system follows a single-agent loop pattern:

- User submits a query
- Initial context gathered from knowledge base (RAG)
- LLM decides: search_knowledge, web_search, or answer
- Tool results added to context
- Context compacted when exceeding 6000 characters
- Loop continues until model answers or max iterations reached

API Endpoint

```
POST /chat/agentic
Request: {"message": "...", "max_iterations": 5}
Response: {"answer": "...", "sources_consulted": [...], "iterations_used": 3}
```

Tools Available

- search_knowledge: Query the RAG knowledge base
- web_search: Search the web for external information
- calculator: Mathematical computations
- get_current_datetime: Current date and time

4. Context Engineering Technique

Technique: Compaction

The ContextManager class (app/agent/context_manager.py) implements compaction. After each iteration of the agentic loop, when accumulated tool results exceed 6,000 characters, the LLM itself is prompted to distill the raw search outputs into a concise summary of key findings.

Problem It Solves

Without compaction, each agentic iteration appends full search results (often 1,000+ tokens each). After 3-4 iterations, the conversation history saturates the model's context window, causing it to lose earlier findings or produce degraded decisions. Compaction preserves essential information while reclaiming context space, allowing the loop to run for more iterations without quality degradation.

Example

If the agent searches the knowledge base for "Python best practices" (800 tokens of results) and then searches the web for "2024 Python standards" (1,200 tokens), compaction reduces these 2,000 tokens into a ~200-token summary of key facts before the next iteration.

5. Agentic Pattern

Pattern: Single-Agent Loop

The cross-source verification task uses a single-agent loop because:

- No context isolation needed: All agents would share the same query and findings
- No parallelization benefit: Sources are checked sequentially because each decision depends on what was found in the previous step
- Specialization not required: A single agent can handle both knowledge base queries and web searches
- Avoiding the sequential bottleneck: Adding agent coordination would introduce communication overhead without reducing the fundamental sequential dependency

Design Decision

A single-agent design is the better fit because the loop's complexity comes from the decision logic (search more vs. answer), not from needing multiple parallel workstreams. Multi-agent would add coordination cost without benefit for this specific task.

6. Evaluation Harness

Metrics Measured

- Task Completion Rate: How often the agent produces a substantive answer
- Tool-Call Correctness: Whether the agent selects appropriate tools
- Trajectory Length: Number of iterations per query
- Failure Classification: Hard (no answer), Soft (inadequate), Cascading Soft (tool failure led to wrong path)

Test Queries (3 Representative)

Query ID	Difficulty	Completed	Tools OK	Iters	Tokens	Duration
simple_01	simple	Yes	Yes	3	4,580	30.5s
moderate_01	moderate	Yes	Yes	1	1,067	7.6s
complex_01	complex	Yes	Yes	3	4,419	18.6s

Summary Metrics

Task Completion Rate: 3/3 (100%)

Tool Correctness Rate: 2/3 (67%)

Average Trajectory Length: 2.3 iterations

Total Tokens Used: 10,066 (prompt: 9,376, completion: 690)

7. Sample Agentic Output

Query: What is Retrieval-Augmented Generation (RAG)?

Iterations: 3 | Sources: search_knowledge, web_search | Tokens: 4,580

Step-by-Step Trajectory:

Iteration 1: search_knowledge

-> Retrieved 5 chunks from knowledge base (rag_guide.md, python_best_practices.txt)

Iteration 2: web_search

-> Searched for "What is Retrieval-Augmented Generation RAG"

Iteration 3: answer (confidence: high)

-> Provided comprehensive definition of RAG with components and benefits

Answer Excerpt:

"Retrieval-Augmented Generation (RAG) is an AI architecture and framework that enhances Large Language Models (LLMs) by incorporating relevant external knowledge at inference time. Instead of relying exclusively on the model's static training data, RAG dynamically retrieves relevant chunks of information from an external knowledge base and injects them directly into the LLM's prompt context before generating a response."

8. Failure Injection Tests

Test 1: Web Search Unavailable

Method: Tool returns error message when web_search is called

Result: PASS - Agent fell back to knowledge base only, did not hallucinate web sources

Test 2: Malformed RAG Output

Method: RAG returns empty context

Result: PASS - Agent searched the web instead, did not hallucinate KB references

Test 3: Provider Timeout

Method: Primary provider (vLLM) checked for health, fallback to Gemini

Result: PASS - Fallback to Gemini activated gracefully

Failure Injection Summary

Test	Result	Notes
Web Search Unavailable	PASS	Agent acknowledged limited sources
Malformed RAG Output	PASS	Agent fell back to web search
Provider Timeout	PASS	Graceful degradation to fallback

9. Additional Requirements

Skill vs. Agent

The cross-source verification capability could not have been implemented as a Skill because Skills are static instruction documents loaded into context. They cannot execute tools, evaluate intermediate results, or make branching decisions at runtime. The agentic feature fundamentally requires the model to call tools, inspect their outputs, and decide whether to call more tools or provide an answer. This dynamic decision-making loop is the defining characteristic of an agent, not a skill.

Token and Cost Accounting

The evaluation harness records `prompt_tokens` and `completion_tokens` for every query. For the agentic loop, average token usage per query is approximately:

- Simple queries: ~4,500 tokens (3 LLM calls)
- Moderate queries: ~1,000 tokens (1 LLM call)
- Complex queries: ~4,400 tokens (3 LLM calls)

The agentic loop adds approximately 2-4x more tokens per query compared to the single-pass `/chat` endpoint, which uses ~800-1,200 tokens per query. This additional cost is the price of iterative verification.

Tool vs. Agent Boundary

The web search tool is modeled as a bounded tool call. It is a stateless, single-step operation: given a query string, it returns search results and terminates. It has no internal state, no multi-step reasoning, and no memory across calls.

In contrast, the agentic loop itself is modeled as an agent because it maintains state across iterations (accumulated findings, context manager state, iteration count) and makes sequential decisions based on intermediate results.

10. Deliverables

Source Code

- app/agent/loop.py - Agentic cross-source verification loop
- app/agent/context_manager.py - Context compaction technique
- app/main.py - FastAPI endpoints including /chat/agent
- app/evaluation/harness.py - Custom evaluation framework
- app/evaluation/test_queries.py - 10 test queries
- app/evaluation/failure_injection.py - Failure injection tests

Architecture Diagram

The agentic loop follows this flow:

User -> Query -> Initialize Context -> Search Knowledge Base -> Context Manager ->
LLM Decision -> (search_knowledge | web_search | answer) -> Response

See README.md for Mermaid diagram.

Evaluation Results

- Task Completion Rate: 100% (3/3)
- Tool Correctness Rate: 67% (2/3)
- Average Trajectory Length: 2.3 iterations
- Total Tokens Used: 10,066
- Failure Injection: 3/3 tests passed